Assignment statements

Lecture 5

Based on Slides of Dr. Norazah Yusof

Assignment statement

- In Java, the assignment statement is used to change the value of a variable
- The equal sign (**=**) is used as the assignment operator
- An assignment statement consists of a variable on the left side of the operator, and an *expression* on the right side of the operator

```
Variable = Expression;
```

  - An *expression* consists of a variable, number, or mix of variables, numbers, operators, and/or method invocations. For example:

```
temperature = 98.6;
count = numberOfBeans;
```

- The assignment operator is automatically executed from right-to-left, so assignment statements can be chained

```
number2 = number1 = 3;
```

Assignment compatibility

- In general, the value of **one type cannot be stored** in a variable of another type.
- A `double` value cannot be stored in an `int` variable.
- An `int` cannot be assigned to a variable of type `boolean`, nor can a `boolean` be assigned to a variable of type `int`
- Example:

```
int intVariable = 2.99; //Illegal
```

- However, there are exceptions to this

```
double doubleVariable = 2;
```

  - For example, an `int` value can be stored in a `double` type

Assignment compatibility

- What would happen if you assign an integer literal to a byte variable?

```
byte b = 3;
byte c = 8;
byte d = 500;
```

Assignment compatibility

- What would happen if you assign an integer literal to a byte variable?

```
byte b = 3; //No problem, 3 fits in a byte
byte c = 8; // No problem, 8 fits in a byte
byte d = 500; // 500 does not fit a byte
```

C:\JavaProg\ByteTest.java:3: possible loss of precision
found : int
required: byte
** byte a = 500;**

Type casting

- Casting lets you convert primitive values from one type to another.
- Two type of casting:
  1. Implicit casting – the conversion happens automatically
  2. Explicit casting – programmer tells the compiler the type to cast
- Example of implicit cast:

```
int a = 100;
long b = a; // Implicit cast, an int value always
            //fits in a long
```

- Example of explicit cast:

```
double d = 4.5;
int i;
i = (int)d; // The value of variable i is 4
            // The value of variable d is 4.5
  //Explicit cast, the integer could lose info
```

Widening conversion

- putting a smaller thing (say, a byte) into a bigger container (like an int).
- an implicit cast happens when you're doing a widening conversion :

  small-value-into-large-container.
- Integer values may be assigned to a double variable without explicit casting, because any integer value can fit in a 64-bit double.
  - Example:

```
double d = 100L; // Implicit cast
```

Narrowing conversion

- put a larger thing (say, a `long`) into a smaller container (like a `int`).
- The large-value-into-small-container conversion requires explicit cast.
- An explicit *type cast* is required to assign a value of larger type (e.g., **`float`**) to a variable with smaller value type (e.g. **`int`**)
  - Example:

```
float a = 100.001f;
int b = (int)a; // Explicit cast, the float
                //could lose info
```

Assignment compatibility

- More generally, a value of any type in the following list can be assigned to a variable of any type that appears to the right of it

  `byte`→`short`→`int`→`long`→`float`→`double`

  `char` ↗

  - Note that as your move down the list from left to right, the range of allowed values for the types becomes larger

Selftest

- Is the following legal/illegal?

```
1. int x = 3957.229;
2. long s = 56L; byte b = (byte)s;
3. byte a = 128;
```

Selftest

- Is the following legal/illegal?

```
1. int x = 3957.229; //compilation error
2. long s = 56L; byte b = (byte)s; //OK
3. byte a = 128; // byte can only hold up to 127
```

Arithmetic operators and expressions

- As in most languages, *expressions* can be formed in Java using variables, constants, and arithmetic operators.
- Java has five (5) arithmetic operators.

Operator	Meaning	Type	Example
+	Addition	Binary	total = cost + tax;
-	Subtraction	Binary	cost = total – tax;
*****	Multiplication	Binary	tax = cost * rate;
/	Division	Binary	salePrice = original / 2;
%	Modulus	Binary	remainder = value % 5;

Shorthand assignment statement

- Shorthand assignment notation combines the *assignment operator* (**=**) and an *arithmetic operator*
- It is used to change the value of a variable by adding, subtracting, multiplying, or dividing by a specified value
- The general form is

```
Variable Op = Expression
```

which is equivalent to

```
Variable = Variable Op (Expression)
```

  - The **`Expression`** can be another variable, a constant, or a more complicated expression
  - Some examples of what **`Op`** can be are **`+`**, **`-`**, **`*`**, **`/`**, or **`%`**

Shorthand assignment statement

Example:	Equivalent To:
`count += 2;`	`count = count + 2;`
`sum -= discount;`	`sum = sum - discount;`
`bonus *= 2;`	`bonus = bonus * 2;`
`time /= rushFactor;`	`time = time / rushFactor;`
`change %= 100;`	`change = change % 100;`
`amount *= count1 + count2;`	`amount = amount * (count1 + count2);`

Arithmetic Operators and Expressions

Promotion

- If an arithmetic operator is combined with **`int`** operands, then the resulting type is **`int`**
- If an arithmetic operator is combined with one or two **`double`** operands, then the resulting type is **`double`**
- If different types are combined in an expression, then the resulting type is the right-most type on the following list that is found within the expression

  `byte`→**`short`**→**`int`**→**`long`**→**`float`**→**`double`**

  `char`

  - Exception: If the type produced should be **`byte`** or **`short`** (according to the rules above), then the type produced will actually be an **`int`**

Arithmetic Operators and Expressions

- An expression can be *fully parenthesized* in order to specify exactly what subexpressions are combined with each operator
- If some or all of the parentheses in an expression are omitted, Java will follow *precedence* rules to determine, in effect, where to place them
  - However, it's best (and sometimes necessary) to include them

Precedence rule

- Parentheses rule: Evaluate expressions in parentheses separately. Evaluate nested parentheses from the inside out.
- Operator precedence rule: Operators in the same expression are evaluated in the order determined by their precedence (from the highest to the lowest).

Operator	*Precedence*
method call	*highest precedence*
`-` (unary)	
`new`, type cast	
`*`, `/`, `%`	
`+`, `-` (binary)	
`=`	*lowest precedence*

- Left associative rule: Operators in the same expression and at the same precedence level are evaluated in left-to-right order.

Precedence rule

- When two operations have equal precedence, the order of operations is determined by *associativity* rules
  - Unary operators of equal precedence are grouped right-to-left
    - **+-+rate** is evaluated as **+(-(+rate))**
  - Binary operators of equal precedence are grouped left-to-right
    - **base + rate + hours** is evaluated as
    - **(base + rate) + hours**
  - Exception: A string of assignment operators is grouped right-to-left
    - **n1 = n2 = n3;** is evaluated as **n1 = (n2 = n3);**

Precedence rule

Evaluation of z – (a + b / 2) * w / y

z - (a + b / 2) * w / y	*Operator, reason evaluated*
--/---	*/, parens and precedence*
---+-------	*+, parens*
----------*--	**, precendence, left assoc.*
----------/--	*/, precedence*
-- - ----------	-

Increment and decrement operator

Operator	Name	Description
++var	preincrement	The expression (++var) increments var by 1 and evaluates to the *new* value in var *after* the increment.
var++	postincrement	The expression (var++) evaluates to the *original* value in var and increments var by 1.
--var	predecrement	The expression (--var) decrements var by 1 and evaluates to the *new* value in var *after* the decrement.
var--	postdecrement	The expression (var--) evaluates to the *original* value in var and decrements var by 1.

Increment and decrement operator

- If `n` is equal to `2`, then `2*(++n)` evaluates to `6`
- If `n` is equal to `2`, then `2*(n++)` evaluates to `5`
- Using increment and decrement operators makes expressions short, but it also makes them complex and difficult to read.
- Avoid using these operators in expressions that modify multiple variables, or the same variable for multiple times such as this: int k = ++i + i.